



**RÉPUBLIQUE ALGÉRIENNE DÉMOCRATIQUE ET
POPULAIRE**



MINISTÈRE DE L'ENSEIGNEMENT SUPÉRIEUR ET DE LA RECHERCHE SCIENTIFIQUE

Université des Sciences et de la Technologie Houari Boumediene

Faculté d'Informatique

Spécialité : Réseaux et Système distribués (RSD)

Techniques d'optimisation

Réseaux

**Processus de Convergence vers un Équilibre de Charge Basé sur l'Équilibre de
Nash**

Présenté par :

**M.BELLABIOD Yanis
M.MEROUANI Rabie
M.BEN SALMA Badreddine
M.GHEDAIFI Ahmed
Mlle.CHERIF Sara**

Devant :

Mme.DAHMANI KHELIFA Djamila

M1-RSD/2025-2026

Introduction Générale

Contexte : L'infrastructure à l'épreuve de l'explosion du trafic

Le paysage des technologies de l'information subit une mutation structurelle profonde. L'omniprésence des architectures Cloud, la transition généralisée vers les microservices, l'essor de la 5G et le déploiement massif d'applications à forte intensité de données ont propulsé le trafic réseau mondial vers des sommets sans précédent. Cette augmentation exponentielle de la demande ne se traduit pas seulement par un besoin de bande passante brute ; elle impose une pression critique sur les infrastructures sous-jacentes.

Les centres de données (*Data Centers*) modernes et les nœuds d'*Edge Computing* se retrouvent au cœur de cette dynamique. Pour garantir une qualité de service (QoS) conforme aux exigences de disponibilité et de latence, la simple mise à l'échelle matérielle (*scale-up*) s'avère insuffisante et économiquement non viable. La clé de voûte de la résilience de ces systèmes distribués réside désormais dans l'efficacité des mécanismes d'allocation des ressources et, plus précisément, dans l'équilibrage de charge (*Load Balancing*). Répartir les flux de données de manière intelligente n'est plus une option d'optimisation, c'est une condition essentielle à la survie des réseaux face à la congestion.

Problématique : Les limites du contrôle centralisé

L'enjeu fondamental de l'équilibrage de charge peut se résumer à une question simple mais algorithmiquement complexe : **comment répartir de façon optimale N utilisateurs sur M serveurs disposant de capacités de traitement et de caractéristiques réseau distinctes ?**

Traditionnellement, cette tâche est confiée à des orchestrateurs centraux qui appliquent des politiques déterministes comme le *Round Robin* ou le *Least Connection*. Bien que robustes dans des environnements prévisibles, ces approches centralisées montrent de graves limites aujourd'hui :

- Elles constituent un point unique de défaillance (*Single Point of Failure*) et un goulot d'étranglement potentiel.
- Elles ignorent la nature hétérogène des exigences des utilisateurs (certains flux priorisent une latence ultra-faible, tandis que d'autres exigent une bande passante massive).

Approche : La Théorie des Jeux comme paradigme d'auto-organisation

Pour contourner les rigidités de la centralisation, une rupture de paradigme s'impose : concevoir le réseau comme un écosystème dynamique d'agents autonomes et rationnels. C'est ici que la **Théorie des Jeux** offre un cadre formel puissant, notamment à travers le modèle des **Jeux de Congestion** (*Congestion Games*).

Dans cette approche, chaque utilisateur devient un *joueur* guidé par une fonction de coût individuelle tri-critère combinant la latence, la congestion et la bande passante. Le but de chaque agent est de minimiser sa propre fonction :

$$C_{ij} = \alpha \cdot d_{ij} + \beta \cdot \left(\frac{L_j}{cap_j} \right) + \gamma \cdot \left(\frac{1}{bw_{ij}} \right) \quad (1)$$

Lorsque ces décisions individuelles non-coopératives se stabilisent, le système atteint un **Équilibre de Nash**. L'écart d'efficacité entre cette situation d'anarchie de routage et une répartition idéale (l'Optimum Social) est évalué par le **Prix de l'Anarchie** (*Price of Anarchy – PoA*) :

$$PoA = \frac{CotTotal'equilibredeNash}{CotTotal'OptimumSocial} \quad (2)$$

Plan du rapport

Pour mener à bien cette étude, le présent rapport est structuré en trois chapitres principaux :

- **Le Chapitre 1 (État de l'Art & Spécification des Besoins)** dresse un panorama des solutions classiques d'équilibrage de charge et introduit les fondements théoriques des jeux de congestion appliqués aux réseaux.
- **Le Chapitre 2 (Conception)** détaille la modélisation mathématique de notre système distribué, la formulation de la fonction de coût et l'implémentation algorithmique des dynamiques de meilleure réponse (*Best-Response Dynamics*).
- **Le Chapitre 3 (Implémentation & Réalisation)** expose l'environnement technologique de la plateforme de simulation développée, baptisée **Shadow Core**, et valide le comportement du modèle face aux scénarios de charge et de pannes.

Contents

Introduction Générale	1
1 État de l’Art & Spécification des Besoins	6
1.1 État de l’Art	6
1.1.1 Équilibre de charge : les approches classiques	6
1.1.2 Limites structurelles des approches centralisées	7
1.1.3 Théorie des jeux appliquée aux réseaux	7
1.1.4 Concepts clés et définitions formelles	8
1.1.5 Travaux existants et outils de simulation	9
1.2 Spécification des Besoins	10
1.2.1 Identification des Acteurs	10
1.2.2 Besoins Fonctionnels	10
1.2.3 Besoins Non Fonctionnels	12
2 Conception	13
2.1 Modélisation Mathématique	13
2.1.1 Formulation du Jeu de Congestion	13
2.1.2 La Fonction de Coût Tri-Critère	14
2.1.3 Définition de l’Équilibre de Nash Pur	14
2.1.4 Définition de l’Optimum de Pareto et de l’Optimum Social	15
2.1.5 Le Prix de l’Anarchie (Price of Anarchy – PoA)	15
2.2 Conception Algorithmique	16

2.2.1	Dynamique de Meilleure Réponse (Best-Response Dynamics) et Détection de Cycles	16
2.2.2	Équilibre de Nash Mixte via la Réponse Quantale (Softmax QRE)	17
2.2.3	Recherche de l'Optimum Social par Amélioration Stricte	17
2.2.4	Formulation des Métriques et Indicateurs Réseau (KPI)	18
2.3	Architecture Logicielle	19
2.3.1	Architecture Modulaire (Modèle / Solveurs / Visualisation)	19
2.3.2	Diagramme de Cas d'Utilisation (Use Case)	20
2.3.3	Diagramme de Séquence	20
2.3.4	Diagramme d'Activité	22
3	Implémentation & Réalisation	25
3.1	Environnement de Développement	25
3.1.1	Choix de l'Architecture Technique	25
3.1.2	Technologies et Bibliothèques Implémentées	25
3.1.3	Logique Algorithmique et Processus de Convergence	26
3.1.4	Implémentation des Algorithmes et Solveurs	26
3.1.5	Architecture Logicielle de l'Interface de Simulation	34
	Conclusion Générale	37

List of Figures

2.1	Diagramme de cas d'utilisation du simulateur Shadow Core	20
2.2	Diagramme de Séquence du Solveur Nash Pur	21
2.3	Diagramme d'activité global du processus de convergence	23
3.1	Implémentation logicielle de la fonction de coût tri-critère	27
3.2	Implémentation logicielle de de la dynamique de meilleure réponse (Nash Pur)	28
3.3	Implémentation logicielle du solveur de Nash Mixte par Softmax stochastique	29
3.4	Implémentation logicielle du solveur d'optimisation de la frontière de Pareto	30
3.5	Implémentation logicielle du solveur d'Équilibre de Réponse Quantale (QRE)	31
3.6	Implémentation logicielle du solveur de recherche locale de l'Optimum Social	32
3.7	Implémentation logicielle du calcul de la fonction de potentiel exact de Rosenthal	33
3.8	Mécanisme logiciel de rééquilibrage dynamique en temps réel	34
3.9	Panneau de configuration tri-critère et d'administration de l'infrastructure réseau	35
3.10	Visualisation topologique des flux et routages dynamiques des agents	36
3.11	Tableau de bord statistique, métriques live du <i>PoA</i> et module d'extraction CSV	36

Chapter 1

État de l'Art & Spécification des Besoins

1.1 État de l'Art

L'essor des architectures distribuées modernes a transformé la gestion des infrastructures réseau en un défi algorithmique permanent. Face à des volumes de requêtes toujours plus denses et imprévisibles, la manière dont nous répartissons la charge de travail conditionne directement la viabilité d'un système. Cette section dresse un panorama critique des méthodes traditionnelles d'équilibrage de charge, en met en lumière les limites intrinsèques, avant d'introduire la théorie des jeux comme une alternative disruptive et élégante pour modéliser le comportement de flux autonomes.

1.1.1 Équilibre de charge : les approches classiques

L'équilibrage de charge (*Load Balancing*) traditionnel repose sur un principe fondamental : un équipement centralisé — qu'il s'agit d'un commutateur matériel ou d'un reverse-proxy logiciel — intercepte l'ensemble des flux entrants pour les distribuer vers un pool de serveurs. Pour opérer cette répartition, trois grandes familles de politiques déterministes prédominent historiquement :

- **Round Robin (et sa variante pondérée Weighted Round Robin)** : C'est l'approche la plus simple et la plus symétrique. Les requêtes sont assignées aux serveurs de manière séquentielle, l'une après l'autre, sous forme de boucle circulaire. Dans sa version pondérée (*Weighted*), l'administrateur attribue un poids fixe à chaque machine en fonction de sa puissance CPU ou de sa mémoire, permettant d'envoyer proportionnellement plus de flux vers les nœuds les plus robustes.
- **Least Connection (Moindres Connexions)** : Plus dynamique, cette politique surveille en temps réel l'état du pool. Lorsqu'une nouvelle requête arrive, le répartiteur l'aiguille vers le serveur qui maintient actuellement le plus petit nombre de sessions actives. Cette approche s'avère particulièrement efficace lorsque les requêtes ont des durées de traitement très hétérogènes.
- **Approches basées sur le hachage (IP Hash)** : Ici, une fonction de hachage applique un calcul mathématique sur l'adresse IP source du client pour déterminer de manière statique le serveur

cible. Cela garantit une certaine persistance : un même utilisateur sera systématiquement redirigé vers le même nœud, facilitant la gestion des sessions locales.

1.1.2 Limites structurelles des approches centralisées

Bien que ces mécanismes aient prouvé leur efficacité dans les architectures traditionnelles, le passage à l'échelle requis par le Cloud, l'Edge Computing et les infrastructures 5G met en évidence trois failles conceptuelles majeures.

Premièrement, l'orchestrateur centralisé incarne par définition un **point unique de défaillance** (*Single Point of Failure – SPoF*). Si le répartiteur subit une avarie matérielle ou une attaque par déni de service (DDoS), l'intégralité du système distribué devient inaccessible, quelle que soit la disponibilité des serveurs en arrière-plan. De plus, à mesure que le nombre N d'utilisateurs grimpe, le traitement et l'analyse de chaque paquet par un unique nœud transforment ce dernier en un goulot d'étranglement (*bottleneck*) critique, augmentant artificiellement la latence globale.

Deuxièmement, ces politiques souffrent d'une forme de **rigidité aveugle**. Elles considèrent les flux de données comme des entités passives et homogènes. En réalité, le trafic moderne est composite : une session de chirurgie à distance ou un flux de véhicule connecté (V2X) exige une latence ultra-faible, tandis qu'une sauvegarde de base de données requiert une bande passante massive mais tolère des délais. Les algorithmes classiques peinent à arbitrer dynamiquement ces exigences contradictoires à grande échelle.

Enfin, l'infrastructure subit le phénomène d'**asymétrie d'information**. Pour prendre une décision parfaite, un orchestrateur central doit interroger en permanence l'état de chaque serveur. Dans un système à forte dynamique, le temps que ces métriques remontent au répartiteur, la réalité du terrain a déjà changé. L'orchestrateur prend alors des décisions basées sur un état passé, ce qui peut provoquer des effets de bord catastrophiques, comme l'effondrement d'un serveur sous un pic soudain de requêtes.

1.1.3 Théorie des jeux appliquée aux réseaux

Face aux blocages de la centralisation, l'idée directrice consiste à inverser la logique : plutôt que de contraindre les flux par le haut, pourquoi ne pas laisser les flux choisir eux-mêmes leur route de manière autonome ? Ce changement de paradigme trouve son assise mathématique dans la théorie des jeux, qui étudie les interactions stratégiques entre décideurs rationnels.

Le réseau n'est plus vu comme une machine rigide, mais comme un marché ou un espace de compétition. Les paquets ou les utilisateurs deviennent des **agents (ou joueurs)** capables d'évaluer leur environnement et de prendre la décision qui maximise leur propre intérêt. Appliquée au routage et à l'équilibrage de charge, cette approche s'appuie sur une littérature académique riche et rigoureuse :

- **Les Jeux de Congestion de Rosenthal (1973)** : Robert Rosenthal a formalisé une classe de jeux magnifiquement adaptée aux réseaux. Dans un jeu de congestion, l'utilité (ou le coût) qu'un joueur

retire du choix d'une ressource (un serveur, une route) dépend exclusivement du nombre total de joueurs choisissant cette même ressource. L'apport fondamental de Rosenthal est d'avoir prouvé que ces jeux possèdent toujours une *fonction de potentiel*. Cette fonction agit comme une boussole globale : chaque fois qu'un utilisateur change individuellement de serveur pour réduire son propre coût, la fonction de potentiel décroît. Cela garantit mathématiquement que le système finira par se stabiliser, excluant les risques d'oscillations chaotiques infinies.

- **Le Selfish Routing de Roughgarden et Tardos :** Ces travaux étudient les réseaux sous l'angle du choix d'itinéraire égoïste (*Selfish Routing*). En analysant des flux massifs, ils ont démontré comment l'absence de coordination pousse les individus à saturer les axes principaux, illustrant des situations paradoxales (comme le paradoxe de Braess) où l'ouverture d'une nouvelle route ou l'ajout d'un serveur peut paradoxalement détériorer le temps de transport global pour tout le monde.
- **L'apport de Milchtaich (1996) sur l'hétérogénéité :** Le modèle de base de Rosenthal supposait des joueurs identiques. Igal Milchtaich a étendu cette théorie aux jeux de congestion avec des **joueurs hétérogènes**, où chaque agent possède sa propre perception du coût (par exemple, un joueur sensible uniquement au débit et un autre sensible uniquement à la latence). Cette extension est cruciale pour les réseaux de télécommunications actuels et constitue le fondement logique de notre simulateur.

1.1.4 Concepts clés et définitions formelles

Pour analyser scientifiquement le comportement de notre système décentralisé, trois concepts économiques et mathématiques sont indispensables :

L'Équilibre de Nash

Un profil de stratégies (une distribution des utilisateurs sur les serveurs) est un **Équilibre de Nash** si aucun joueur ne peut réduire son coût en modifiant unilatéralement son choix. Autrement dit, à l'équilibre, chaque agent formule une *Meilleure Réponse* (*Best-Response*) aux choix des autres. Personne n'a de regret, la configuration est stable. C'est l'état naturel vers lequel converge un réseau laissé à l'anarchie de ses utilisateurs.

L'Optimum de Pareto et l'Optimum Social

Une situation est qualifiée d'**Optimum de Pareto** s'il est impossible d'améliorer la situation (réduire le coût) d'un utilisateur sans détériorer celle d'au moins un autre. L'**Optimum Social**, quant à lui, va plus loin en cherchant l'efficacité collective maximale : c'est la configuration qui minimise la *somme globale* de tous les coûts du réseau. C'est la répartition idéale qu'un ingénieur réseau ou un orchestrateur omniscient chercherait à imposer par la force pour optimiser son infrastructure.

Le Prix de l'Anarchie (Price of Anarchy – PoA)

Introduit par Koutsoupias et Papadimitriou, le **Prix de l'Anarchie** est le ratio qui mesure la perte d'efficacité due au manque de coordination. Il quantifie mathématiquement le coût de l'égoïsme :

$$PoA = \frac{CotGlobal'quilibredeNash(goste)}{CotGlobal'OptimumSocial(Idal)} \quad (1.1)$$

Si le $PoA = 1$, cela signifie que la liberté individuelle mène à une solution parfaite pour la communauté. Plus le PoA s'éloigne de 1, plus le réseau souffre de l'absence de contrôle centralisé, légitimant la mise en place de mécanismes d'incitation ou de régulation.

1.1.5 Travaux existants et outils de simulation

L'évaluation empirique de ces concepts en environnement réel se heurte à des coûts de déploiement prohibitifs, rendant l'usage de simulateurs indispensable. Dans la littérature et l'industrie, plusieurs outils se partagent le secteur, chacun répondant à des philosophies distinctes :

- **NS-3 (Network Simulator 3) :** C'est la référence académique pour la simulation de réseaux de paquets. Ultra-précis au niveau des couches physiques et de transport (implémentation fidèle des piles TCP/IP, routage BGP, etc.), il s'avère cependant extrêmement lourd à configurer, nécessite des temps de calcul longs, et n'intègre pas nativement de solveurs orientés théorie des jeux ou d'interfaces cinétiques interactives en temps réel.
- **Gambit :** Un outil logiciel puissant dédié exclusivement au calcul d'équilibres en théorie des jeux (matrices de gains, arbres de décision). S'il est parfait pour résoudre des jeux abstraits à petite échelle, il est totalement déconnecté des réalités techniques des réseaux (incapable de simuler une topologie de serveurs, des débits ou des pannes de liens).
- **Simulateurs SDN et Émulateurs (Mininet) :** Très utilisés pour tester des politiques d'équilibrage de charge au sein de réseaux pilotés par logiciel (*Software-Defined Networking*). Bien qu'ancrés dans le réel, ils demandent une infrastructure de virtualisation complète et s'avèrent peu adaptés pour faire de l'exploration mathématique pure ou de la pédagogie visuelle immédiate.

C'est précisément à l'intersection de ces outils que se positionne notre solution, **Shadow Core v12**. Là où NS-3 est trop lourd et Gambit trop abstrait, notre plateforme propose un environnement léger, visuel et mathématiquement rigoureux, capable de simuler en temps réel la dynamique cinétique des flux hétérogènes tout en résolvant instantanément les indicateurs macroscopiques comme le Prix de l'Anarchie.

1.2 Spécification des Besoins

La transition d'un modèle mathématique abstrait vers un outil logiciel opérationnel exige une définition claire et structurée des objectifs à atteindre. La spécification des besoins permet de traduire les concepts de la théorie des jeux en fonctionnalités logicielles concrètes, tout en cernant les contraintes techniques indispensables pour garantir la pertinence de l'outil. Cette section détaille les acteurs du système, les exigences fonctionnelles au cœur de la simulation, ainsi que les critères de qualité non fonctionnels qui font de notre plateforme une solution robuste.

1.2.1 Identification des Acteurs

L'architecture de notre simulateur a été pensée pour répondre aux attentes de trois catégories distinctes d'utilisateurs, chacun exploitant la plateforme avec des objectifs spécifiques :

- **Les Étudiants en Informatique et Systèmes Distribués (RSD) :** Pour ce public, l'outil agit comme un pont entre la théorie abstraite et la réalité des infrastructures. L'étudiant utilise la plateforme pour visualiser concrètement des notions complexes comme la convergence d'un algorithme, l'impact de la surcharge d'un lien réseau ou la différence palpable entre une optimisation égoïste et une gestion centralisée.
- **Les Enseignants en Théorie des Jeux et Réseaux :** L'acteur enseignant cherche avant tout un support didactique interactif pour illustrer ses cours magistraux. Le simulateur lui permet de générer à la volée des scénarios de jeu (homogènes, hétérogènes, cas de pannes) et de valider visuellement devant un auditoire les théorèmes de point fixe ou l'évolution fine du Prix de l'Anarchie (*PoA*).
- **Les Chercheurs et Concepteurs d'Architectures Réseau :** Cet acteur utilise l'outil comme un environnement de prototypage rapide. Il cherche à ajuster finement les paramètres de la fonction de coût (α, β, γ) pour observer comment de nouvelles politiques de routage décentralisées se comportent avant de les implémenter sur des émulateurs plus lourds ou des infrastructures réelles.

1.2.2 Besoins Fonctionnels

Les besoins fonctionnels décrivent les actions que le système doit être capable d'exécuter pour répondre aux attentes des utilisateurs. Ils s'articulent autour de quatre grands axes :

1. Modélisation paramétrable du système distribué

Le système doit permettre de configurer et d'instancier un écosystème réseau hautement hétérogène. L'utilisateur doit pouvoir :

Déployer un nombre dynamique M de serveurs, chacun défini par sa capacité de traitement (cap), sa latence de base (d) et sa bande passante nominale (bw).

Générer un flux continu ou un ensemble de N clients atomiques caractérisés par des poids spécifiques (w) représentant leur charge de trafic, ainsi que des matrices de préférences individuelles.

Manipuler l'environnement en temps réel, notamment par la désactivation instantanée d'un nœud pour simuler une panne réseau majeure.

2. Implémentation des solveurs théoriques

Le cœur calculatoire de l'application doit intégrer trois moteurs de résolution distincts :

Solveur Équilibre de Nash Pur : Calcul de la distribution stable des flux via une dynamique de meilleure réponse (*Best-Response Dynamics*). Ce moteur doit intégrer un mécanisme robuste de détection de cycles pour éviter le verrouillage du calcul en cas d'oscillations infinies.

Solveur Équilibre de Nash Mixte : Modélisation d'agents à rationalité limitée ou indécis, en s'appuyant sur une distribution probabiliste de type *Softmax* (Quantal Response Equilibrium), permettant d'introduire un facteur de bruit ou d'incertitude dans les choix des flux.

Solveur Optimum Social (Pareto) : Détermination de l'allocation idéale centralisée qui minimise le coût total cumulé de l'infrastructure, servant de point de référence absolu pour mesurer l'efficacité collective.

3. Visualisation cinétique et graphique

L'application ne doit pas se contenter d'afficher des tableaux de chiffres statiques. Elle doit impérativement projeter une animation fluide (cinétique) qui illustre physiquement la migration des flux d'un serveur à un autre lors des phases de calcul d'équilibre. De plus, elle doit intégrer des graphiques dynamiques (courbes de convergence, diagrammes de charge) mis à jour en temps réel.

4. Extraction et calcul des métriques macroscopiques

Pour chaque scénario évalué, le système doit calculer instantanément des indicateurs clés de performance (KPI) :

Le **Prix de l'Anarchie** (PoA) pour quantifier la perte d'efficacité due à l'égoïsme.

L'**Indice d'Équité de Jain** pour mesurer si la charge est répartie de manière équitable entre les infrastructures disponibles.

Le **Makespan** et le taux d'utilisation global du réseau.

1.2.3 Besoins Non Fonctionnels

Les besoins non fonctionnels définissent les exigences de qualité, de performance et d'ergonomie qui conditionnent la robustesse et l'adoption de l'application.

- **Performance et Réactivité Temporelle (Calcul Rapide) :** Les algorithmes de recherche d'équilibre (Nash et Pareto) doivent s'exécuter en tâche de fond de manière quasi-instantanée (idéalement en quelques millisecondes pour des configurations standard). Le temps de calcul ne doit en aucun cas bloquer le thread principal de l'interface graphique, garantissant un rendu à 60 images par seconde (FPS).
- **Ergonomie et Intuitivité de l'Interface :** L'application doit adopter une charte graphique moderne, épurée et immersive (inspirée des interfaces de supervision réseau à haute visibilité). L'accès aux commandes critiques (lancement du flux, pannes provoquées, choix du solveur) doit être direct, limitant la courbe d'apprentissage pour les utilisateurs.
- **Portabilité et Légèreté (Architecture Client-Side) :** Afin de garantir une accessibilité maximale sans barrière technique, le simulateur doit s'exécuter entièrement côté client (*Single Page Application*) au sein d'un simple navigateur web standard (Chrome, Firefox, Safari). L'absence de dépendance à un serveur back-end ou à une base de données lourde élimine les contraintes de déploiement et facilite son utilisation immédiate en salle de cours ou en conférence.
- **Robustesse Numérique et Tolérance aux Anomalies :** Le moteur mathématique doit être protégé contre les instabilités numériques courantes dans les simulations distribuées. Cela implique une gestion stricte des divisions par zéro (par exemple, si la capacité résiduelle d'un serveur tend vers le néant) et le traitement des valeurs indéterminées (*NaN*), assurant la continuité de l'expérience utilisateur même sous des configurations de stress-test extrêmes.

Chapter 2

Conception

La phase de conception constitue la passerelle entre l'analyse théorique des besoins et la réalisation logicielle. Elle a pour but de formaliser mathématiquement les briques du système distribué et de structurer l'architecture des algorithmes avant leur codage. Ce chapitre expose d'abord le modèle formel du jeu de congestion hétérogène et les critères d'équilibre, avant de détailler la conception algorithmique et structurelle de la plateforme *Shadow Core*.

2.1 Modélisation Mathématique

Pour concevoir un simulateur fidèle aux réalités opérationnelles, nous devons traduire le réseau en un modèle mathématique rigoureux. Nous formalisons notre problème d'équilibrage de charge sous la forme d'un **Jeu de Congestion Atomique et Hétérogène**. L'hétérogénéité s'exprime ici à double niveau : les serveurs possèdent des capacités techniques distinctes, et les utilisateurs (flux) ont des exigences et des volumes de trafic asymétriques.

2.1.1 Formulation du Jeu de Congestion

Un jeu de congestion se définit formellement par un triplet $\Gamma = (\mathcal{N}, \mathcal{M}, (C_i)_{i \in \mathcal{N}})$, où :

- **Les Joueurs ($\mathcal{N}$)** : Un ensemble fini de N utilisateurs ou flux de données autonomes, notés $\mathcal{N} = \{1, 2, \dots, N\}$. Chaque joueur i est caractérisé par un poids strict $w_i \in \mathbb{R}^+$, qui représente le volume de trafic ou la charge de requêtes par seconde qu'il injecte dans le système.
- **Les Stratégies ($\mathcal{M}$)** : Un ensemble de M ressources partagées représentant les serveurs physiques ou virtuels disponibles, notés $\mathcal{M} = \{1, 2, \dots, M\}$. Pour chaque joueur i , l'espace des stratégies disponibles consiste à choisir un serveur unique $j \in \mathcal{M}$ pour y router l'intégralité de sa charge w_i .
- **La Charge des Serveurs (L_j)** : Pour un profil de stratégie global $S = (s_1, s_2, \dots, s_N)$ où $s_i \in \mathcal{M}$ désigne le choix du joueur i , la charge totale cumulée L_j subie par un serveur j est la somme des

poids des joueurs ayant sélectionné ce serveur :

$$L_j(S) = \sum_{i \in \mathcal{N}: s_i=j} w_i \quad (2.1)$$

2.1.2 La Fonction de Coût Tri-Critère

Dans les applications distribuées modernes, la performance d'un serveur ne peut se résumer à une seule métrique. Un utilisateur évaluant la qualité d'un nœud Cloud prend en compte la réactivité, la surcharge et la bande passante. C'est pourquoi nous concevons une **fonction de coût tri-critère personnalisée**.

Pour un joueur i ayant choisi le serveur j , son coût individuel C_{ij} est défini par la formulation suivante :

$$C_{ij}(S) = \alpha \cdot d_{ij} + \beta \cdot \left(\frac{L_j(S)}{cap_j} \right) + \gamma \cdot \left(\frac{1}{bw_{ij}} \right) \quad (2.2)$$

Où les paramètres décrivent l'état de l'infrastructure et la sensibilité de l'application :

d_{ij} représente la latence de propagation ou le délai de communication de base entre le client i et le serveur j .

cap_j est la capacité de traitement maximale nominale du serveur j . Le ratio $\frac{L_j}{cap_j}$ exprime le facteur de congestion de la ressource. Plus le serveur est saturé, plus ce coût explose, modélisant les files d'attente CPU/Réseau.

bw_{ij} désigne la bande passante disponible sur le lien reliant le client au serveur. Le coût étant inversement proportionnel au débit ($\frac{1}{bw}$), le modèle pénalise logiquement les connexions lentes.

$\alpha, \beta, \gamma \geq 0$ sont les coefficients de pondération globale du réseau. Ils permettent de calibrer la simulation selon le profil des flux : un réseau orienté IoT priorisera la latence (α élevé), tandis qu'un réseau de stockage Cloud priorisera la bande passante (γ élevé).

Chaque joueur cherche rationnellement à **minimiser** cette fonction de coût, ce qui équivaut à maximiser sa propre utilité perçue $U_{ij} = \frac{1}{1+C_{ij}}$.

2.1.3 Définition de l'Équilibre de Nash Pur

Dans un contexte non coopératif, les joueurs prennent leurs décisions de manière isolée et égoïste. Le système converge naturellement vers un état de stabilité macroscopique appelé **Équilibre de Nash Pur**.

Formellement, un profil de décision $S^* = (s_1^*, s_2^*, \dots, s_N^*)$ constitue un Équilibre de Nash Pur si, pour tout joueur $i \in \mathcal{N}$ et pour toute stratégie alternative $s_i \in \mathcal{M}$, nous avons :

$$C_{i,s_i^*}(s_i^*, S_{-i}^*) \leq C_{i,s_i}(s_i, S_{-i}^*) \quad (2.3)$$

Où S_{-i}^* représente le vecteur des choix de tous les autres joueurs à l'exception du joueur i .

Interprétation humaine : Cette équation signifie qu'à l'équilibre, aucun utilisateur ne peut réduire son coût de routage par une déviation unilatérale. Si un joueur décide de changer de serveur alors que les autres restent fixes, sa situation se détériorera ou restera identique. C'est un point fixe du système où la rationalité individuelle engendre une stabilité statique du réseau.

2.1.4 Définition de l'Optimum de Pareto et de l'Optimum Social

L'Équilibre de Nash traduit ce qui se produit lorsque le réseau est laissé à lui-même. Pour mesurer son efficacité, il faut le confronter aux critères d'optimisation centralisés issus de l'économie du bien-être.

L'Optimum de Pareto

Un profil de stratégie S est un **Optimum de Pareto** s'il n'existe aucune autre configuration alternative S' permettant d'améliorer la situation d'un joueur sans dégrader celle d'un autre. Autrement dit :

$$\forall S', \quad (\exists i \in \mathcal{N}, C_i(S') < C_i(S)) \implies (\exists k \in \mathcal{N}, C_k(S') > C_k(S)) \quad (2.4)$$

Sur un réseau, cela signifie que toutes les ressources sont exploitées de telle manière qu'on ne peut plus offrir un meilleur chemin à un paquet sans pénaliser directement un autre paquet du système.

L'Optimum Social

L'Optimum Social est un cas particulier, plus strict, de l'optimum de Pareto. Il représente la configuration idéale recherchée par un administrateur système omniscient. Son but est de minimiser le **coût social global** du réseau, défini comme la somme de tous les coûts individuels :

$$S^{social} = \arg \min_S \sum_{i=1}^N C_i(S) \quad (2.5)$$

À l'Optimum Social, on peut tout à fait sacrifier l'intérêt d'un joueur particulier (lui imposer un serveur plus lent) si cela permet de désengorger massivement le reste de l'infrastructure et de réduire drastiquement le coût total cumulé de la communauté.

2.1.5 Le Prix de l'Anarchie (Price of Anarchy – PoA)

Le conflit fondamental entre l'égoïsme individuel (Nash) et la planification centralisée (Optimum Social) est quantifié par le **Prix de l'Anarchie** (PoA). Il s'exprime par le ratio suivant :

$$PoA = \frac{\sum_{i=1}^N C_i(S^*)}{\sum_{i=1}^N C_i(S^{social})} \quad (2.6)$$

Puisque l'Optimum Social est par définition la configuration minimale possible pour la collectivité, nous avons toujours $PoA \geq 1$.

- Si $PoA = 1$, l'anarchie est parfaite : les choix égoïstes des utilisateurs décentralisés aboutissent exactement au même niveau d'efficacité qu'une infrastructure entièrement contrôlée par un orchestrateur.
- Si $PoA > 1$, le ratio mesure le gaspillage de ressources réseau provoqué par le manque de coordination des flux. Par exemple, un $PoA = 1.30$ signifie que l'égoïsme des agents engendre une dégradation de 30 % des performances globales de l'infrastructure par rapport à une répartition optimale.

2.2 Conception Algorithmique

Le passage de la formalisation mathématique à l'exécution logicielle nécessite la conception d'algorithmes de résolution robustes. Cette section détaille la logique interne des trois moteurs de calcul de notre simulateur — la dynamique de meilleure réponse pour l'équilibre pur, l'approche stochastique pour l'équilibre mixte, et l'exploration de l'optimum collectif —, ainsi que la formulation des indicateurs de performance réseau (KPI) chargés d'évaluer la qualité des configurations obtenues.

2.2.1 Dynamique de Meilleure Réponse (Best-Response Dynamics) et Détection de Cycles

Pour déterminer l'Équilibre de Nash Pur de manière décentralisée, nous implémentons l'algorithme cinétique de la *Best-Response Dynamics*. Le principe imite le comportement itératif d'agents autonomes : à chaque cycle, le système passe en revue les joueurs les uns après les autres. Chaque joueur examine l'état actuel de congestion des serveurs et évalue si une migration unilatérale vers un autre nœud réduit sa fonction de coût personnelle.

Cependant, dans un jeu discret avec des joueurs hétérogènes (disposant de poids différents), la convergence mathématique n'est pas automatique et peut se heurter à des oscillations infinies. Par exemple, deux joueurs lourds peuvent se pourchasser indéfiniment entre deux serveurs, créant un effet de "ping-pong" algorithmique. Pour garantir la robustesse du simulateur, nous avons conçu un double mécanisme de protection :

1. **Le Seuil de Tolérance ($GAIN_THRESHOLD$)** : Un joueur ne valide sa migration vers un nouveau serveur que si le gain de coût obtenu dépasse un certain seuil minimal strict ($C_{actuel} - C_{hypothétique} >$

ε). Cela simule l'inertie ou le coût de migration dans un vrai réseau et élimine les micro-oscillations non significatives.

2. **La Détection de Cycles par Empreinte d'État (State Hashing) :** Avant de permettre à un joueur de se déplacer, l'algorithme capture la matrice de routage globale actuelle (l'affectation exacte de chaque flux à son serveur) et en calcule une signature unique (un *hash*). Cette empreinte est stockée dans une structure de données à accès rapide (*Visited States Set*). Si l'algorithme génère une configuration d'allocation déjà rencontrée par le passé, un cycle est immédiatement détecté. Le solveur interrompt alors proprement la simulation pour éviter une boucle infinie, garantissant l'arrêt du programme.

2.2.2 Équilibre de Nash Mixte via la Réponse Quantale (Softmax QRE)

Dans la réalité des infrastructures, les utilisateurs ou les routeurs ne disposent pas toujours d'une information parfaite, ou peuvent faire preuve d'une rationalité limitée (sensibilité au bruit, imprécisions de mesure de la latence). Pour modéliser ce comportement probabiliste, nous concevons un solveur basé sur l'Équilibre de Réponse Quantale (*Quantal Response Equilibrium – QRE*), qui s'appuie sur une loi de probabilité *Softmax*.

Au lieu de choisir de façon déterministe et stricte le serveur le moins coûteux, chaque joueur i attribue une probabilité de sélection P_{ij} à chaque serveur j . Cette probabilité est inversement proportionnelle au coût attendu sur ce nœud, modulée par un paramètre de température τ (ou facteur de rationalité) :

$$P_{ij} = \frac{\exp\left(-\frac{C_{ij}}{\tau}\right)}{\sum_{k=1}^M \exp\left(-\frac{C_{ik}}{\tau}\right)} \quad (2.7)$$

Analyse cinétique du comportement :

- Lorsque la température $\tau \rightarrow 0$, les joueurs deviennent infiniment rationnels et stricts : la probabilité converge vers 1 pour le meilleur serveur et 0 pour les autres, rejoignant la dynamique de la *Best-Response* pure.
- Lorsque $\tau \rightarrow \infty$, le bruit prend le dessus : les joueurs ignorent complètement les niveaux de congestion et choisissent leurs serveurs de manière totalement uniforme ($P_{ij} = \frac{1}{M}$). Ce solveur mixte permet d'étudier la résilience des protocoles de routage face à des comportements d'utilisateurs imprévisibles ou fluctuants.

2.2.3 Recherche de l'Optimum Social par Amélioration Stricte

Contrairement aux deux solveurs précédents qui simulent l'anarchie et l'égoïsme, le solveur de l'Optimum Social (qui est mathématiquement un Optimum de Pareto global spécifique) adopte la perspective d'un

planificateur centralisé. Son objectif est de découvrir la distribution des flux qui minimise la fonction de coût collectif $\sum C_i$.

Pour y parvenir de manière efficace sans subir l'explosion combinatoire liée au nombre de configurations possibles (M^N), l'algorithme procède par **améliorations strictes successives** (*Local Search Optimization*) :

L'orchestrateur teste virtuellement le déplacement de chaque joueur vers chaque serveur alternatif.

Contrairement au mode égoïste, le déplacement n'est validé **que si et seulement si** la somme de tous les coûts de la communauté diminue après la migration, même si le coût individuel du joueur déplacé augmente temporairement.

L'algorithme boucle jusqu'à ce qu'aucune migration possible ne puisse plus réduire le coût social global, garantissant l'obtention d'un minimum local fort, très proche ou égal à l'optimum global absolu.

2.2.4 Formulation des Métriques et Indicateurs Réseau (KPI)

Pour évaluer et comparer scientifiquement les configurations générées par nos différents solveurs, notre architecture logicielle intègre le calcul systématique de trois métriques standard de l'ingénierie des systèmes distribués :

1. L'Indice d'Équité de Jain (Jain's Fairness Index)

Cet indice permet de mesurer si la charge de travail ou le coût est réparti de manière équitable entre les différentes ressources de calcul, évitant qu'un serveur soit totalement saturé pendant que ses voisins restent oisifs. Il est formulé de la sorte :

$$\mathcal{J}(L_1, L_2, \dots, L_M) = \frac{\left(\sum_{j=1}^M L_j\right)^2}{M \cdot \sum_{j=1}^M L_j^2} \quad (2.8)$$

L'indice de Jain est strictement borné : $\mathcal{J} \in \left[\frac{1}{M}, 1\right]$. Une valeur de 1 indique une équité parfaite (tous les serveurs supportent exactement la même charge L_j). Une valeur proche de $\frac{1}{M}$ indique une distribution catastrophique où une unique machine encaisse l'intégralité du trafic du réseau.

2. Le Makespan

Emprunté à la théorie de l'ordonnancement, le Makespan représente le temps total nécessaire pour que l'infrastructure traite l'ensemble des requêtes en cours. Dans notre modèle, il correspond graphiquement à la charge du serveur le plus lourdement sollicité du système :

$$Makespan = \max_{j \in \mathcal{M}} L_j \quad (2.9)$$

Minimiser le Makespan est un objectif critique en ingénierie réseau, car le maillon le plus lent ou le plus chargé détermine la réactivité maximale de l'ensemble de la plateforme.

3. Le Taux d'Utilisation Global

Cette métrique macroscopique mesure l'efficacité globale de l'allocation des ressources matérielles en comparant la charge totale injectée à la capacité de traitement cumulée du pool de serveurs :

$$U_{global} = \frac{\sum_{j=1}^M L_j}{\sum_{j=1}^M cap_j} \times 100\% \quad (2.10)$$

Un taux d'utilisation équilibré (proche de 70 % à 80 %) atteste d'un dimensionnement idéal de l'infrastructure. Si ce taux approche des 100 %, le réseau est en état de sous-dimensionnement critique, tandis qu'un taux trop bas traduit un gaspillage économique des ressources Cloud déployées.

2.3 Architecture Logicielle

La transition d'un modèle d'optimisation théorique vers une application fluide et interactive nécessite une structuration logicielle rigoureuse. Afin de garantir la modularité, la maintenabilité et l'évolutivité de la plateforme *Shadow Core*, nous avons opté pour une architecture entièrement découplée, documentée à l'aide du langage de modélisation unifié (UML). Cette section détaille l'organisation des modules applicatifs ainsi que les diagrammes comportementaux et structurels du système.

2.3.1 Architecture Modulaire (Modèle / Solveurs / Visualisation)

Bien que le simulateur s'exécute intégralement côté client dans un navigateur web, son code source n'est pas monolithique. Il est structuré selon un paradigme proche du motif *Modèle-Vue-Contrôleur* (MVC), divisant l'application en trois couches logiques indépendantes :

- **La Couche Modèle (Data & State) :** Ce module gère l'état brut de la simulation. Il encapsule les structures de données représentant les entités physiques et logiques du réseau : l'ensemble des serveurs (avec leurs attributs de capacité, latence et bande passante) et l'ensemble des clients/joueurs (avec leurs poids et profils de sensibilité α, β, γ). Cette couche est purement descriptive et ne contient aucune logique de décision.
- **La Couche Solveurs (Moteurs de Calcul) :** C'est le cœur algorithmique indépendant de l'application. Il regroupe les fonctions de calcul de coût, le moteur de hachage d'état pour la détection de cycles, et les trois algorithmes de résolution (*Nash Pure*, *Nash Mixed*, *Social Optimum*). Ce module interroge le Modèle, exécute les calculs de déviation stratégique ou de probabilités, puis met à jour la matrice d'allocation des flux.

- **La Couche Visualisation et Rendu (Vue) :** Ce module est responsable de l'expérience utilisateur et de l'affichage cinétique. Il s'appuie sur l'API *HTML5 Canvas* pour dessiner l'animation des flux migrant en temps réel d'un nœud à un autre, et sur la bibliothèque *Chart.js* pour tracer les courbes de convergence des métriques (PoA, indice de Jain, Makespan). Elle communique avec le modèle en "lecture seule" pour rafraîchir l'interface à haute fréquence (60 FPS) sans interférer avec les boucles de calcul.

2.3.2 Diagramme de Cas d'Utilisation (Use Case)

Le diagramme de cas d'utilisation formalise les interactions entre nos acteurs cibles (étudiants, enseignants, chercheurs) et les grandes fonctionnalités offertes par l'interface de la plateforme.

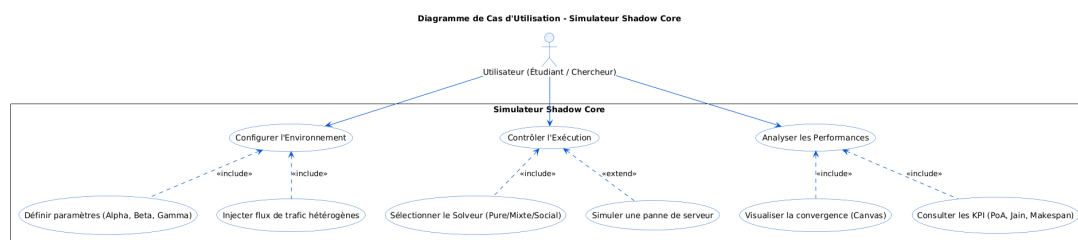


Figure 2.1: Diagramme de cas d'utilisation du simulateur Shadow Core

Comme l'illustre la Figure 2.1, l'utilisateur principal interagit avec le système à travers trois modules fonctionnels majeurs :

1. **La configuration de l'environnement** (initialisation des paramètres réseaux α, β, γ , injection de flux hétérogènes, dimensionnement du pool de serveurs).
2. **Le contrôle de l'exécution** (sélection du solveur théorique, lancement de la simulation cinétique, déclenchement instantané de pannes sur les nœuds).
3. **L'analyse des performances** (observation visuelle des trajectoires de convergence, consultation du Prix de l'Anarchie et des rapports d'équité).

2.3.3 Diagramme de Séquence

Le diagramme de séquence formalise la chronologie des échanges de messages et d'appels de fonctions entre la couche visuelle, l'orchestrateur de simulation et les solveurs lors d'un cycle d'optimisation de type *Best-Response Dynamics*.

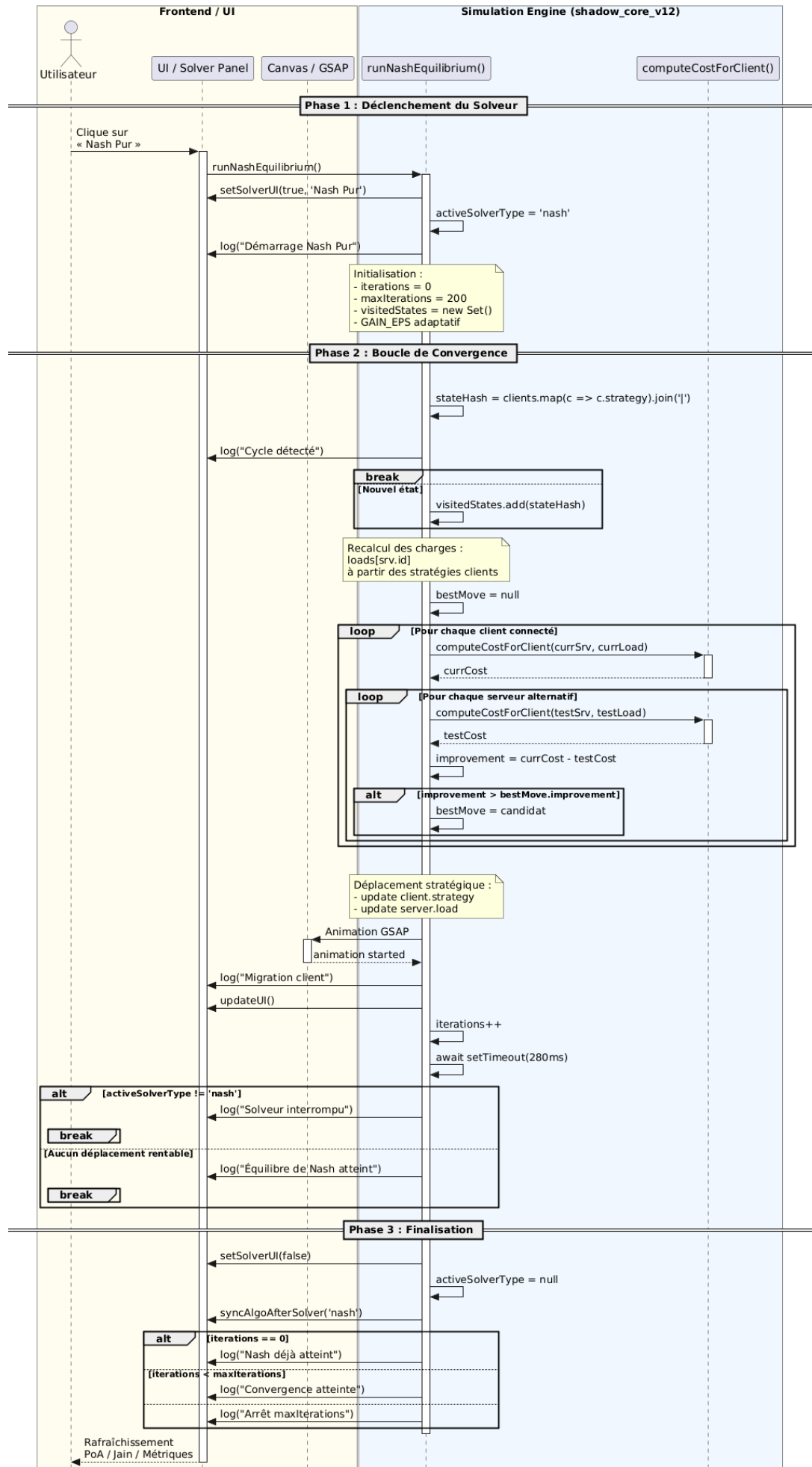


Figure 2.2: Diagramme de Séquence du Solveur Nash Pur

La cinématique des messages suit un ordre séquentiel strict (voir Figure 2.2) :

- a. L'utilisateur sélectionne le mode « Équilibre de Nash » depuis l'interface graphique (*UI Layer*) ce qui déclenche la configuration du routage.
- b. L'interface invoque le solveur principal `runNashEquilibrium()`, responsable de l'exécution de la dynamique de convergence vers l'équilibre de Nash.
- c. Le solveur capture l'état courant des stratégies des clients via une empreinte de type : `clients.map(c => c.targetServerId).join(',')` afin de détecter les oscillations stratégiques et vérifier si l'état appartient déjà à l'ensemble `visitedStates`.
- d. Le solveur parcourt ensuite l'ensemble des joueurs et évalue, pour chaque serveur alternatif, le coût hypothétique à l'aide de la fonction `computeCostForClient()`. Le gain potentiel est calculé comme la différence entre le coût courant et le coût simulé après migration.
- e. Lorsque le meilleur gain identifié dépasse le seuil `GAIN_EPS`, le solveur applique une déviation unilatérale en modifiant la stratégie du joueur concerné (`client.targetServerId`), puis recalcule les charges des serveurs.
- f. Une notification de mise à jour est ensuite transmise à la couche de rendu afin de déclencher les animations Canvas/GSAP et le rafraîchissement des métriques visuelles (PoA, indice de Jain, matrices de charge et graphiques Chart.js).
- g. Le processus itératif se poursuit jusqu'à atteindre un état stable où aucun joueur ne peut améliorer individuellement son utilité, correspondant ainsi à un équilibre de Nash pur.

2.3.4 Diagramme d'Activité

Le diagramme d'activité décrit le flux de contrôle interne et la logique décisionnelle du processus de calcul d'équilibre, mettant en exergue les structures conditionnelles, les boucles d'itérations et les points d'arrêt de sécurité.

Diagramme d'Activité — Processus de Convergence vers l'Équilibre de Nash

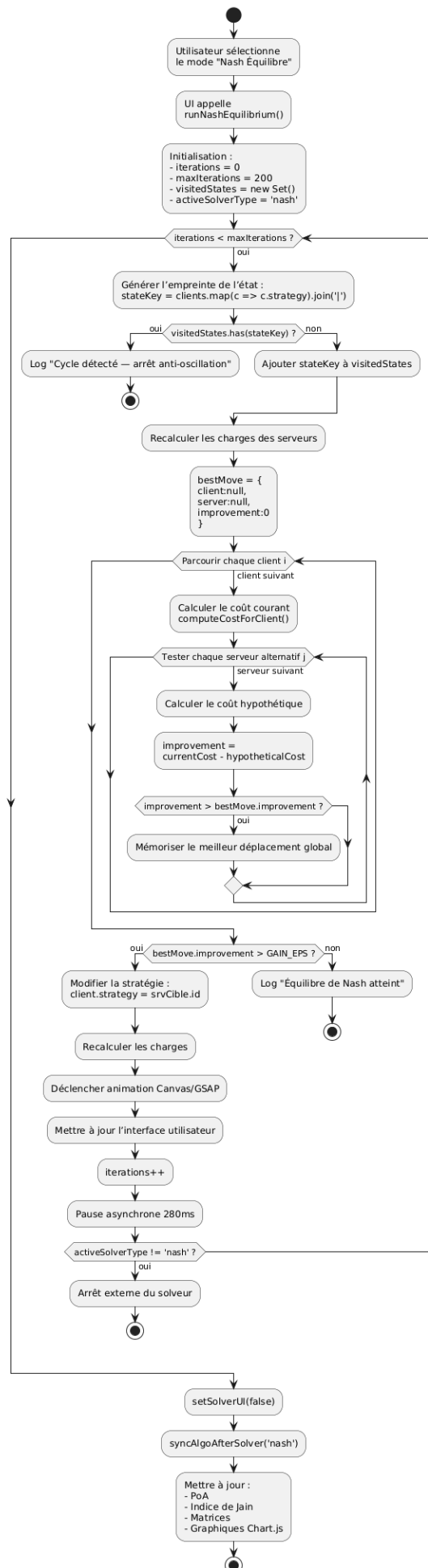


Figure 2.3: Diagramme d'activité global du processus de convergence

Le traitement du solveur suit une logique algorithmique itérative précise (voir Figure 2.3). Dès l'activation du mode « Nash Équilibre », le système invoque la fonction `runNashEquilibrium()`, responsable de la dynamique de convergence vers un équilibre de Nash pur. Une boucle principale est alors exécutée tant que le nombre maximal d'itérations (`iterations < maxIterations`) n'est pas atteint.

Au début de chaque itération, le solveur génère une empreinte représentant l'état courant des stratégies des joueurs à partir des allocations des clients vers les serveurs. Cette empreinte est ensuite comparée à l'ensemble `visitedStates` afin de détecter d'éventuels cycles d'exécution. Si l'état courant a déjà été rencontré, le système considère qu'un phénomène d'oscillation stratégique est survenu et interrompt immédiatement le processus afin d'éviter toute boucle infinie.

Dans le cas contraire, l'empreinte est enregistrée dans l'historique des états visités, puis le solveur recalcule les charges actuelles des serveurs avant d'entamer le parcours séquentiel des agents. Pour chaque joueur, le système évalue le coût courant associé à sa stratégie actuelle ainsi que les coûts hypothétiques obtenus en migrant vers les différents serveurs alternatifs. Le gain potentiel est calculé comme la différence entre le coût courant et le coût simulé après déviation unilatérale.

À l'issue de cette phase d'évaluation, le solveur conserve uniquement le meilleur déplacement global produisant l'amélioration la plus importante.

Si ce gain dépasse le seuil minimal `GAIN-EPS`, la stratégie du joueur concerné est mise à jour, les charges des serveurs sont recalculées, puis l'interface graphique est synchronisée afin de déclencher les animations Canvas GSAP ainsi que le rafraîchissement des métriques visuelles.

Chapter 3

Implémentation & Réalisation

Ce chapitre décrit en détail l’environnement technique, les algorithmes implémentés et l’interface utilisateur du simulateur SHADOW_CORE. Cette plateforme de simulation interactive, basée sur la théorie des jeux pour l’équilibrage de charge en réseau, intègre dans cette version révisée trois nouveaux solveurs (QRE, Optimum Social Local, Potentiel de Rosenthal), deux nouvelles politiques de routage (IP Hash, Weighted Round Robin), ainsi qu’un scénario d’étude dédié au Paradoxe de Braess reposant sur une refonte complète de l’interface graphique.

3.1 Environnement de Développement

3.1.1 Choix de l’Architecture Technique

L’application SHADOW_CORE v13 est développée sous la forme d’une application web monofichier (*Single File Application*) autonome de près de 4 000 lignes de code. Toute la logique métier (modélisation réseau, solveurs de théorie des jeux, boucle d’animation et couche de rendu) s’exécutent entièrement côté client au sein du navigateur web. Cette architecture élimine le besoin d’un serveur applicatif *backend* ou d’un outil de *build* complexe, garantissant ainsi une portabilité, une légèreté et une autonomie maximales du simulateur.

3.1.2 Technologies et Bibliothèques Implémentées

Le développement repose exclusivement sur l’utilisation de standards web natifs, complétés par des bibliothèques spécialisées injectées via des réseaux de diffusion de contenu (CDN). L’ensemble des outils technologiques employés est synthétisé dans le Tableau 3.1.

Technologie / Librairie	Rôle au sein du Simulateur	Source / Version
HTML5 / CSS3	Structuration et mise en page <i>responsive</i> de l'interface	Natif
JavaScript (ES2020)	Logique métier, moteurs des solveurs et gestion des états	Natif
Canvas API 2D	Rendu graphique temps réel (nœuds, clients, flux de paquets)	Natif
Chart.js	Génération des graphiques statistiques (coût social, convergence)	CDN (jsDelivr)
GSAP 3.12	Interpolation et animations fluides lors des déviations de joueurs	CDN (cdnjs)
MathJax 3	Rendu dynamique des formules mathématiques de routage (LaTeX)	CDN (jsDelivr)
Google Fonts	Intégration des typographies (<i>Rajdhani</i> , <i>Orbitron</i> , <i>IBM Plex Mono</i>)	CDN (Google)

Table 3.1: Synthèse de l'environnement technologique de SHADOW_CORE

3.1.3 Logique Algorithmique et Processus de Convergence

Le traitement du solveur suit une logique algorithmique itérative précise. Dès l'activation du mode Nash Équilibre, le système invoque la fonction `runNashEquilibrium()`, responsable de la dynamique de convergence vers un équilibre de Nash pur. Une boucle principale est alors exécutée tant que le nombre maximal d'itérations (`iterations` est inférieur à `maxIterations`) n'est pas atteint.

Au début de chaque itération, le solveur génère une empreinte représentant l'état courant des stratégies des joueurs à partir des allocations des clients vers les serveurs. Cette empreinte est ensuite comparée à l'ensemble `visitedStates` afin de détecter d'éventuels cycles d'exécution. Si l'état courant a déjà été rencontré, le système considère qu'un phénomène d'oscillation stratégique est survenu et interrompt immédiatement le processus afin d'éviter toute boucle infinie.

Dans le cas contraire, l'empreinte est enregistrée dans l'historique des états visités, puis le solveur recalcule les charges actuelles des serveurs avant d'entamer le parcours séquentiel des agents. Pour chaque joueur, le système évalue le coût courant associé à sa stratégie actuelle ainsi que les coûts hypothétiques obtenus en migrant vers les différents serveurs alternatifs. Le gain potentiel est calculé comme la différence entre le coût courant et le coût simulé après déviation unilatérale.

À l'issue de cette phase d'évaluation, le solveur conserve uniquement le meilleur déplacement global produisant l'amélioration la plus importante. Si ce gain dépasse le seuil minimal `GAIN_EPS`, la stratégie du joueur concerné est mise à jour, les charges des serveurs sont recalculées, puis l'interface graphique est synchronisée afin de déclencher les animations Canvas/GSAP ainsi que le rafraîchissement des métriques visuelles.

3.1.4 Implémentation des Algorithmes et Solveurs

La simulation implémente un jeu de congestion à n joueurs hétérogènes, défini formellement par les composantes suivantes :

- **Ensemble de joueurs (I)** : L'ensemble fini de joueurs est défini par $I = \{1, \dots, n\}$, où chaque entité correspond à un `ClientAgent` possédant un rang de priorité spécifique parmi les classes S, A, B, C, E .
- **Ensemble de stratégies (S_j)** : Pour chaque joueur, l'espace des stratégies admissibles correspond

à l'ensemble des serveurs actifs du réseau.

- **Fonction de coût individuelle (C_{ij})** : L'évaluation se traduit par une fonction de coût $C_{ij} : S \rightarrow R^+$.
- **Profil de stratégies (σ)** : L'état global du réseau est défini par le vecteur de profil $\sigma = (\sigma_1, \dots, \sigma_n) \in \prod_i S_i$.

Chaque joueur i possède un poids strict $w_i \in \{1, \dots, 10\}$. La charge effective supportée par un serveur j donné est alors calculée par la formule suivante :

$$L_j = \sum_{i: \sigma_i=j} w_i \quad (3.1)$$

Algorithme 1 — Fonction de Coût Tri-Critère

Cet algorithme représente le cœur mathématique du système et permet d'évaluer le coût $C_{ij}(\sigma)$ associé à chaque agent :

$$C_{ij}(\sigma) = \alpha \cdot d_{ij} + \beta \cdot \left(\frac{L_j}{cap_j} \right) + \gamma \cdot \left(\frac{1}{bw_{ij} + \varepsilon} \right) \quad (3.2)$$



```
function computeCostForClient(clientId, server, hypotheticalLoad) {
  const latMatrix = networkParams.latency[clientId];
  const lat_raw_ij = (latMatrix && latMatrix[server.id] !== undefined)
    ? latMatrix[server.id]
    : (server.latency !== undefined ? server.latency : 10);
  const d_ij = Math.max(lat_raw_ij, 0.001);

  const bwMatrix = networkParams.bandwidth[clientId];
  const bw_raw_ij = (bwMatrix && bwMatrix[server.id] !== undefined)
    ? bwMatrix[server.id]
    : (server.bandwidth !== undefined ? server.bandwidth : 100);
  const bw_ij = Math.max(bw_raw_ij, 0.1);

  const capacity = Math.max(server.capacity || 1, 0.001);
  const currentLoad = hypotheticalLoad !== undefined ? hypotheticalLoad : (server.load || 0);

  const latencyCost = networkParams.alpha * d_ij;
  const loadCost = networkParams.beta * (currentLoad / capacity);
  const bwCost = networkParams.gamma * (1 / (bw_ij + COST_BW_EPS));

  const cost = latencyCost + loadCost + bwCost;
  return isFinite(cost) && !isNaN(cost) ? cost : 9999;
}
```

Figure 3.1: Implémentation logicielle de la fonction de coût tri-critère

Propriété clé : la fonction présente une monotonie stricte en L_j . Plus un serveur est chargé, plus son coût augmente, ce qui induit naturellement la recherche d'un équilibre.

Algorithme 2 — Équilibre de Nash Pur (Best-Response)

Implémenté au sein de la fonction `runNashEquilibrium()`, ce module exécute une dynamique de meilleure réponse (*Best-Response Dynamics*) dont la convergence est garantie par l'existence d'un potentiel exact défini par :

$$\Phi(\sigma) = \sum_j \sum_{k=0}^{L_j} \frac{\beta \cdot k}{cap_j} \quad (3.3)$$

Pour éviter les phénomènes d'oscillations infinies lorsque la composante de charge domine, le système intègre un epsilon adaptatif ($\epsilon = 0,05$ si $\beta > 1,5\alpha$, sinon $\epsilon = 0,001$).

```

async function runNashEquilibrium() {
  if (solverRunning) return;
  const connectedClients = clients.filter(c => c.strategy !== null);
  if (connectedClients.length === 0) return;

  solverRunning = true;
  activeSolverType = 'nash';

  let iterations = 0;
  const maxIterations = 200;

  // Définition de l'empreinte de l'état (stateHash) et de l'ensemble (visitedStates)
  const visitedStates = new Set();
  const getStateHash = () => clients.map(c => c.strategy || 'null').join('|');

  // Configuration de l'epsilon adaptatif (GAIN_EPS) selon la dominance de charge
  const betaHeavy = networkParams.beta > networkParams.alpha * 1.5;
  const GAIN_EPS = betaHeavy ? 0.05 : 0.001;

  while (iterations < maxIterations) {
    const active = servers.filter(s => s.status === 'OK');
    if (!active.length) break;

    // Vérification de cycle à partir du profil de stratégies courant
    const stateHash = getStateHash();
    if (visitedStates.has(stateHash)) {
      break; // Cycle détecté -> Arrêt forcé
    }
    visitedStates.add(stateHash);

    // Recalcul des charges effectives des serveurs
    const loads = {};
    active.forEach(s => loads[s.id] = 0);
    clients.forEach(c => {
      if (c.strategy && loads[c.strategy] !== undefined) {
        loads[c.strategy] += (c.weight || 1);
      }
    });

    // Recherche séquentielle du meilleur déplacement (Best-Response)
    let bestMove = null;
    for (const client of clients) {
      if (!client.strategy) continue;

      const currSrv = active.find(s => s.id === client.strategy);
      if (!currSrv) continue;
      const currCost = computeCostForClient(client.id, currSrv, loads[currSrv.id]);

      for (const srv of active) {
        if (srv.id === client.strategy) continue;

        const testLoad = (loads[srv.id] || 0) + (client.weight || 1);
        const cost = computeCostForClient(client.id, srv, testLoad);
        const gain = currCost - cost; // gain = C_i(sigma_i) - C_i(j')

        if (gain > GAIN_EPS && (!bestMove || gain > bestMove.improvement)) {
          bestMove = { client, targetServer: srv, improvement: gain };
        }
      }
    }

    // Si aucun gain n'est supérieur à epsilon -> Arrêt car l'équilibre est atteint
    if (!bestMove) break;

    // Application physique du mouvement retenu
    const oldSrv = active.find(s => s.id === bestMove.client.strategy);
    if (oldSrv) {
      oldSrv.load = Math.max(0, (oldSrv.load || 0) - (bestMove.client.weight || 1));
    }
    bestMove.targetServer.load = (bestMove.targetServer.load || 0) + (bestMove.client.weight || 1);
    bestMove.client.strategy = bestMove.targetServer.id;

    iterations++;
    if (activeSolverType !== 'nash') break;
  }

  solverRunning = false;
  activeSolverType = null;
}

```

Figure 3.2: Implémentation logicielle de la dynamique de meilleure réponse (Nash Pur)

La logique de fonctionnement suit les étapes suivantes : tant que le nombre d'itérations est inférieur à 200, l'algorithme vérifie si l'empreinte de l'état (stateHash) appartient à l'ensemble `visitedStates`. Si c'est le cas, un cycle est détecté et le processus s'arrête. Sinon, pour chaque joueur i et pour chaque serveur alternatif $j' \neq \sigma_i$, le gain est évalué par $gain = C_i(\sigma_i) - C_i(j')$. Si le meilleur gain identifié dépasse ϵ , le mouvement est appliqué, sinon le processus s'arrête car l'Équilibre de Nash Pur est atteint.

Algorithme 3 — Nash Mixte par Softmax

La fonction `runMixedNashSolver()` approche l'équilibre mixte de Nash via une distribution probabiliste Softmax paramétrée à une température fixe $\tau = 0,05$. En définissant l'utilité individuelle par $U_i(j) = \frac{1}{1+C_{ij}}$, la probabilité de sélection d'un serveur se formule ainsi :

$$\sigma_i^*(j) = \frac{\exp\left(\frac{U_i(j)}{\tau}\right)}{\sum_{j'} \exp\left(\frac{U_i(j')}{\tau}\right)} \quad (3.4)$$

```

async function runMixedNashSolver() {
  if (solverRunning) return;
  const connectedClients = clients.filter(c => c.strategy !== null);
  if (!connectedClients.length) return;

  solverRunning = true;
  activeSolverType = 'mixed';

  for (let i = 0; i < clients.length; i++) {
    const client = clients[i];
    const active = servers.filter(s => s.status === 'OK');
    if (!active.length) break;

    // Évaluation des charges courantes du système
    const loads = {};
    active.forEach(s => loads[s.id] = 0);
    clients.forEach(c => {
      if (c.strategy && loads[c.strategy] !== undefined)
        loads[c.strategy] += (c.weight || 1);
    });

    // Calcul des utilités individuelles (U = 1 / (1 + Coût))
    const utilities = {};
    active.forEach(srv => {
      const testLoad = (client.strategy === srv.id) ? (loads[srv.id] || 0) : (loads[srv.id] || 0) +
        (client.weight || 1);
      const cost = computeCostForClient(client.id, srv, testLoad);
      utilities[srv.id] = 1 / (1 + cost);
    });

    // Distribution de probabilité Softmax à température fixe (tau = 0.05)
    const tau = 0.05;
    const entries = Object.entries(utilities).map(([id, u]) => ({ id, val: Math.exp(u / tau) }));
    const sumExp = entries.reduce((s, x) => s + x.val, 0);

    // Sélection stochastique cumulative (Méthode de la roulette)
    let r = Math.random(), acc = 0, selectedId = client.strategy;
    for (const item of entries) {
      acc += item.val / sumExp;
      if (r <= acc) { selectedId = item.id; break; }
    }

    // Application du changement de stratégie s'il y a déviation
    if (selectedId !== client.strategy) {
      const oldSrv = servers.find(s => s.id === client.strategy);
      const newSrv = servers.find(s => s.id === selectedId);
      if (oldSrv) oldSrv.load = Math.max(0, (oldSrv.load || 0) - (client.weight || 1));
      if (newSrv) newSrv.load = (newSrv.load || 0) + (client.weight || 1);
      client.strategy = selectedId;
    }
  }

  solverRunning = false;
  activeSolverType = null;
}

```

Figure 3.3: Implémentation logicielle du solveur de Nash Mixte par Softmax stochastique

Algorithme 4 — Optimisation de Pareto

La fonction `runParetoSolver()` recherche une configuration sur la frontière d'efficacité. Un déplacement n'est validé que s'il respecte simultanément quatre contraintes : une amélioration individuelle stricte ($C_i(cible) < C_i(courant) - \epsilon$), la non-dégradation des autres agents ($\forall k \neq i : C_k(aprs) \leq C_k(avant) + \epsilon$), la non-saturation des infrastructures ($L_j + w_i \leq cap_j \times 1,25$), et un gain d'utilité global. L'utilisation d'un $\epsilon = \pm 0,001$ symétrique immunise le traitement contre les erreurs de précision flottante.

```

async function runParetoSolver() {
  if (solverRunning) return;
  const connected = clients.filter(c => c.strategy !== null);
  if (!connected.length) return;

  solverRunning = true;
  activeSolverType = 'pareto';

  const getSystemCosts = (loads) => clients.map(c => {
    if (!c.strategy) return 999;
    const srv = servers.find(s => s.id === c.strategy && s.status === 'OK');
    if (!srv) return 999;
    return computeCostForClient(c.id, srv, loads[srv.id] || 0);
  });

  let steps = 0, improved = true;
  while (improved && steps < 60) {
    improved = false;
    const active = servers.filter(s => s.status === 'OK');

    const stratLoads = {};
    active.forEach(s => stratLoads[s.id] = 0);
    clients.forEach(c => {
      if (c.strategy && stratLoads[c.strategy] !== undefined)
        stratLoads[c.strategy] += (c.weight || 1);
    });

    const currentCosts = getSystemCosts(stratLoads);

    outer: for (let i = 0; i < clients.length; i++) {
      const client = clients[i];
      if (!client.strategy) continue;

      const currSrv = active.find(s => s.id === client.strategy);
      if (!currSrv) continue;

      const currCost = computeCostForClient(client.id, currSrv, stratLoads[currSrv.id] || 0);
      const currUtility = 1 / (1 + currCost);

      for (const candSrv of active) {
        if (candSrv.id === client.strategy) continue;

        const oldLoad = stratLoads[currSrv.id] || 0;
        const newLoad = (stratLoads[candSrv.id] || 0) + (client.weight || 1);

        // Contrainte 3 : Non-saturation critique (seuil 125%)
        if (newLoad > (candSrv.capacity || 1) * 1.25) continue;

        const nextCostSelf = computeCostForClient(client.id, candSrv, newLoad);
        const nextUtility = 1 / (1 + nextCostSelf);

        // Contrainte 1 : Amélioration individuelle stricte
        if (nextCostSelf >= currCost - 0.001 || nextUtility <= currUtility) continue;

        // Génération du profil de coûts théorique après déviation
        const testCosts = clients.map((c, j) => {
          if (j === i) return nextCostSelf;
          if (c.strategy === candSrv.id) return computeCostForClient(c.id, candSrv, newLoad);
          if (c.strategy === currSrv.id) return computeCostForClient(c.id, currSrv, Math.max(0,
            oldLoad - (client.weight || 1)));
        });

        const csrv = servers.find(s => s.id === c.strategy);
        return csrv ? computeCostForClient(c.id, csrv, stratLoads[csrv.id] || 0) : 999;
      });

      // Contrainte 2 : Non-dominance de Pareto (Epsilon symétrique)
      const PARETO_EPS = 0.001;
      const someoneBetter = testCosts.some((c, j) => c < currentCosts[j] - PARETO_EPS);
      const noOneWorse = testCosts.every((c, j) => c <= currentCosts[j] + PARETO_EPS);

      if (someoneBetter && noOneWorse) {
        stratLoads[currSrv.id] = Math.max(0, oldLoad - (client.weight || 1));
        stratLoads[candSrv.id] = newLoad;
        client.strategy = candSrv.id;
        improved = true;
        steps++;

        if (activeSolverType !== 'pareto') { improved = false; }
        break outer;
      }
    }
  }

  solverRunning = false;
  activeSolverType = null;
}

```

Figure 3.4: Implémentation logicielle du solveur d'optimisation de la frontière de Pareto

La fonction `runParetoSolver()` recherche une configuration sur la frontière d'efficacité. Un déplacement

ment n'est validé que s'il respecte simultanément quatre contraintes : une amélioration individuelle stricte ($C_i(cible) < C_i(courant) - \epsilon$), la non-dégradation des autres agents ($\forall k \neq i : C_k(aprs) \leq C_k(avant) + \epsilon$), la non-saturation des infrastructures ($L_j + w_i \leq cap_j \times 1,25$), et un gain d'utilité global. L'utilisation d'un $\epsilon = \pm 0,001$ symétrique immunise le traitement contre les erreurs de précision flottante.

Algorithme 5 — Équilibre QRE (Quantal Response)

```

async function runParetoSolver() {
  if (solverRunning) return;
  const connected = clients.filter(c => c.strategy !== null);
  if (!connected.length) return;

  solverRunning = true;
  activeSolverType = 'pareto';

  const getSystemCosts = (loads) => clients.map(c => {
    if (!c.strategy) return 999;
    const srv = servers.find(s => s.id === c.strategy && s.status === 'OK');
    if (!srv) return 999;
    return computeCostForClient(c.id, srv, loads[srv.id] || 0);
  });

  let steps = 0, improved = true;
  while (improved && steps < 60) {
    improved = false;
    const active = servers.filter(s => s.status === 'OK');

    const stratLoads = {};
    active.forEach(s => stratLoads[s.id] = 0);
    clients.forEach(c => {
      if (c.strategy && stratLoads[c.strategy] !== undefined)
        stratLoads[c.strategy] += (c.weight || 1);
    });

    const currentCosts = getSystemCosts(stratLoads);

    outer: for (let i = 0; i < clients.length; i++) {
      const client = clients[i];
      if (!client.strategy) continue;

      const currSrv = active.find(s => s.id === client.strategy);
      if (!currSrv) continue;

      const currCost = computeCostForClient(client.id, currSrv, stratLoads[currSrv.id] || 0);
      const currUtility = 1 / (1 + currCost);

      for (const candSrv of active) {
        if (candSrv.id === client.strategy) continue;

        const oldLoad = stratLoads[currSrv.id] || 0;
        const newLoad = (stratLoads[candSrv.id] || 0) + (client.weight || 1);

        // Contrainte 3 : Non-saturation critique (seuil 125%)
        if (newLoad > (candSrv.capacity || 1) * 1.25) continue;

        const nextCostSelf = computeCostForClient(client.id, candSrv, newLoad);
        const nextUtility = 1 / (1 + nextCostSelf);

        // Contrainte 1 : Amélioration individuelle stricte
        if (nextCostSelf >= currCost - 0.001 || nextUtility <= currUtility) continue;

        // Génération du profil de coûts théorique après déviation
        const testCosts = clients.map((c, j) => {
          if (j === i) return nextCostSelf;
          if (c.strategy === candSrv.id) return computeCostForClient(c.id, candSrv, newLoad);
          if (c.strategy === currSrv.id) return computeCostForClient(c.id, currSrv, Math.max(0,
            oldLoad - (client.weight || 1)));
          const csrv = servers.find(s => s.id === c.strategy);
          return csrv ? computeCostForClient(c.id, csrv, stratLoads[csrv.id] || 0) : 999;
        });

        // Contrainte 2 : Non-dominance de Pareto (Epsilon symétrique)
        const PARETO_EPS = 0.001;
        const someoneBetter = testCosts.some((c, j) => c < currentCosts[j] - PARETO_EPS);
        const noOneWorse = testCosts.every((c, j) => c <= currentCosts[j] + PARETO_EPS);

        if (someoneBetter && noOneWorse) {
          stratLoads[currSrv.id] = Math.max(0, oldLoad - (client.weight || 1));
          stratLoads[candSrv.id] = newLoad;
          client.strategy = candSrv.id;
          improved = true;
          steps++;

          if (activeSolverType !== 'pareto') { improved = false; }
          break outer;
        }
      }
    }

    solverRunning = false;
    activeSolverType = null;
  }
}

```

Figure 3.5: Implémentation logicielle du solveur d'Équilibre de Réponse Quantale (QRE)

La fonction `runQRESolver()` modélise les joueurs sous l'angle d'une rationalité limitée. Les agents

choisissent une route avec une probabilité décroissante exponentiellement par rapport au coût, gouvernée par la température τ (contrôlée en direct par un composant d'interface) :

$$P_{ij} = \frac{\exp\left(-\frac{C_{ij}}{\tau}\right)}{\sum_k \exp\left(-\frac{C_{ik}}{\tau}\right)} \quad (3.5)$$

Lorsque $\tau \rightarrow 0$, le QRE converge vers un équilibre de Nash pur ; lorsque $\tau \rightarrow \infty$, le choix des stratégies tend vers une distribution uniforme. Le réglage de ce paramètre permet d'observer visuellement la transition entre rationalité parfaite et exploration purement stochastique.

Algorithme 6 — Optimum Social Local

Contrairement aux approches égoïstes (Nash) ou consensuelles (Pareto), la fonction `runSocialLocalOptimum()` accepte la dégradation du coût individuel d'un agent si cette action permet de réduire la somme totale des coûts du système.

```

async function runSocialOptimumLocalSearch() {
  if (solverRunning) return;
  const connected = clients.filter(c => c.strategy !== null);
  if (connected.length) return;

  solverRunning = true;
  activeSolverType = 'pareto';

  const computeTotalSocialCost = (loads) => {
    return clients.reduce((sum, c) => {
      if (!c.strategy) return sum;
      const srv = servers.find(s => s.id === c.strategy && s.status === 'OK');
      if (!srv) return sum;
      return sum + computeCostForClient(c.id, srv, loads[srv.id] || 0) * (c.weight || 1);
    }, 0);
  };

  let improved = true, steps = 0;
  while (improved && steps < 80) {
    improved = false;
    const active = servers.filter(s => s.status === 'OK');

    const loads = {};
    active.forEach(s => loads[s.id] = 0);
    clients.forEach(c => {
      if (c.strategy && loads[c.strategy] !== undefined)
        loads[c.strategy] += (c.weight || 1);
    });

    const currentSocialCost = computeTotalSocialCost(loads);
    let bestMove = null, bestGain = 0;

    // Exploration de l'espace des voisinages pour maximiser le gain social
    for (const client of clients) {
      if (!client.strategy) continue;
      const currSrv = active.find(s => s.id === client.strategy);
      if (!currSrv) continue;

      for (const candSrv of active) {
        if (candSrv.id === client.strategy) continue;

        // Simulation de la déviation collective
        const testLoads = { ...loads };
        testLoads[currSrv.id] = Math.max(0, (testLoads[currSrv.id] || 0) - (client.weight || 1));
        testLoads[candSrv.id] = (testLoads[candSrv.id] || 0) + (client.weight || 1);

        const oldStrategy = client.strategy;
        client.strategy = candSrv.id;
        const newSocialCost = computeTotalSocialCost(testLoads);
        client.strategy = oldStrategy;

        const gain = currentSocialCost - newSocialCost;
        if (gain > 0.001 && gain > bestGain) {
          bestGain = gain;
          bestMove = { client, from: currSrv, to: candSrv, testLoads };
        }
      }
    }

    // Application du mouvement optimal du point de vue social
    if (bestMove) {
      bestMove.from.load = Math.max(0, (bestMove.from.load || 0) - (bestMove.client.weight || 1));
      bestMove.to.load = (bestMove.to.load || 0) + (bestMove.client.weight || 1);
      bestMove.client.strategy = bestMove.to.id;
      improved = true;
      steps++;

      if (activeSolverType !== 'pareto') break;
    }
  }

  solverRunning = false;
  activeSolverType = null;
}

```

Figure 3.6: Implémentation logicielle du solveur de recherche locale de l'Optimum Social

L'algorithme effectue une descente gloutonne sur la fonction objectif sociale :

$$C_{social}(\sigma) = \sum_i C_i(\sigma) \quad (3.6)$$

Ce solveur fournit la valeur de référence stable C^* indispensable au calcul du Prix de l'Anarchie.

Algorithme 7 — Potentiel de Rosenthal

La fonction `computeRosenthalPotential()` calcule à chaque itération la valeur numérique du potentiel exact du système afin d'offrir une preuve rigoureuse de la stabilité et de la convergence de la dynamique de meilleure réponse :

$$\Phi(\sigma) = \sum_j \left[L_j \cdot \left(\alpha \cdot d_j + \frac{\gamma}{bw_j + \epsilon} \right) + \beta \cdot \frac{L_j \cdot (L_j + 1)}{2 \cdot cap_j} \right] \quad (3.7)$$



```
function computeRosenthalPotential() {
  const active = servers.filter(s => s.status === 'OK');
  if (!active.length) return 0;

  let potential = 0;
  active.forEach(srv => {
    const L = Math.round(srv.load || 0);
    const cap = Math.max(srv.capacity || 1, 0.001);
    const d = Math.max(srv.latency || 5, 0.001);
    const bw = Math.max(srv.bandwidth || 100, 0.1);

    const baseCost = networkParams.alpha * d + networkParams.gamma * (1 / (bw + COST_BW_EPS));
    const loadTerm = networkParams.beta * (L * (L + 1) / 2) / cap;

    potential += L * baseCost + loadTerm;
  });

  return isFinite(potential) ? potential : 0;
}
```

Figure 3.7: Implémentation logicielle du calcul de la fonction de potentiel exact de Rosenthal

Rééquilibrage Dynamique en Temps Réel

La fonction `triggerDynamicRebalance()` est invoquée de façon atomique à chaque modification structurelle de l'environnement (ajustement des poids α, β, γ , variation des capacités, pannes matérielles ou modification du nombre de clients). Elle recalcule instantanément la meilleure réponse globale de chaque agent sans déclencher d'artefact visuel, assurant la cohérence continue du système.

```

function triggerDynamicRebalance() {
  const active = servers.filter(s => s.status === 'OK');
  if (!active.length) return;

  // Recalcul instantané du vecteur de charge global
  const loads = {};
  active.forEach(s => loads[s.id] = 0);
  clients.forEach(c => {
    if (c.strategy && loads[c.strategy] !== undefined) loads[c.strategy] += (c.weight || 1);
  });

  clients.forEach(client => {
    // Cas 1 : Réadresse immédiate d'un client orphelin ou déconnecté
    if (!client.strategy) {
      const bestSrv = getBestResponseServer(client, active, loads);
      if (bestSrv) {
        client.strategy = bestSrv.id;
        loads[bestSrv.id] = (loads[bestSrv.id] || 0) + (client.weight || 1);
        bestSrv.load = (bestSrv.load || 0) + (client.weight || 1);
      }
      return;
    }

    // Cas 2 : Évaluation d'une déviation unilatérale (Meilleure Réponse)
    const currSrv = active.find(s => s.id === client.strategy);
    if (!currSrv) { client.strategy = null; return; }

    const currCost = computeCostForClient(client.id, currSrv, loads[currSrv.id]);
    const bestSrv = getBestResponseServer(client, active, loads, currSrv.id);
    if (!bestSrv || bestSrv.id === client.strategy) return;

    const testLoad = (loads[bestSrv.id] || 0) + (client.weight || 1);
    const newCost = computeCostForClient(client.id, bestSrv, testLoad);

    // Validation du déplacement si l'amélioration dépasse le seuil epsilon
    if (newCost < currCost - 0.001) {
      loads[currSrv.id] = Math.max(0, (loads[currSrv.id] || 0) - (client.weight || 1));
      currSrv.load = Math.max(0, (currSrv.load || 0) - (client.weight || 1));
      loads[bestSrv.id] = (loads[bestSrv.id] || 0) + (client.weight || 1);
      bestSrv.load = (bestSrv.load || 0) + (client.weight || 1);
      client.strategy = bestSrv.id;
    }
  });
}

function getBestResponseServer(client, activeServers, loads, excludeId) {
  let best = null, bestCost = Infinity;
  activeServers.forEach(srv => {
    if (excludeId && srv.id === excludeId) return;
    const testLoad = (loads[srv.id] || 0) + (client.weight || 1);
    const cost = computeCostForClient(client.id, srv, testLoad);
    if (cost < bestCost) { bestCost = cost; best = srv; }
  });
  return best;
}

```

Figure 3.8: Mécanisme logiciel de rééquilibrage dynamique en temps réel

3.1.5 Architecture Logicielle de l'Interface de Simulation

Afin de valider expérimentalement les modèles théoriques de routage tri-critère, une plateforme de simulation visuelle interactive a été entièrement développée. Cette application permet d'observer en temps réel la dynamique des agents, l'évolution des fonctions de coût et l'émergence des équilibres.

L'environnement graphique est structuré en plusieurs modules complémentaires, présentés en détail ci-après.

Panneau de Configuration des Paramètres et Scénarios

Le volet gauche de l'interface (voir Figure 3.9) centralise la gestion des variables structurelles et mathématiques du réseau :

- **Politique de routage et sélecteurs de solveurs** : Un menu interactif permet de permuter instantanément entre les cinq algorithmes implémentés (*Nash Pur*, *Nash Mixte*, *QRE Softmax*, *Pareto*, et *Social Local*).
- **Ajustement des poids tri-critères** : Des curseurs dynamiques contrôlent en direct les coefficients d'importance de la latence (α), de la charge/capacité (β), et de l'inverse de la bande passante (γ), redéfinissant immédiatement la fonction objectif C_{ij} .
- **Gestion des infrastructures et pannes (Fault-Tolerance)** : Chaque serveur dispose de curseurs indépendants pour modifier sa capacité ou sa latence propre, ainsi qu'un bouton d'action DÉSACTIVER simulant une rupture ou une panne matérielle immédiate afin de tester la résilience des algorithmes.
- **Gestion du trafic et des clients** : Un module dédié permet de configurer l'intensité globale du flux de paquets ou d'injecter dynamiquement de nouveaux clients en sélectionnant leur profil de priorité (ex. : *Rang B*, *Poids $w = 6$*).

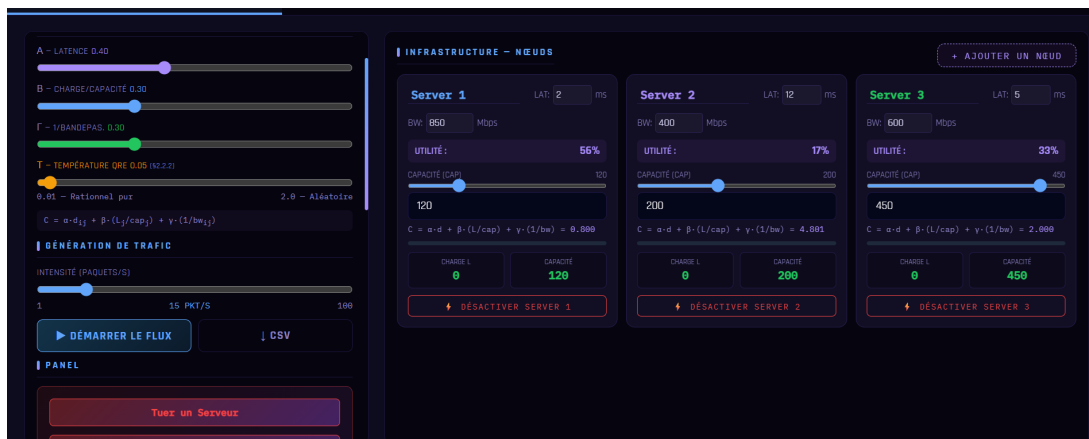


Figure 3.9: Panneau de configuration tri-critère et d'administration de l'infrastructure réseau

Visualisation Cartographique et Monitoring Graphique

Le cœur visuel de la plateforme (illustré par la Figure 3.10) fournit une cartographie topologique en temps réel de l'état distributif du système :

- **Grphe biparti dynamique** : La colonne de gauche matérialise les clients actifs identifiés par leur adresse de session (ex. : CLT-74HHE) tandis que la colonne de droite représente les serveurs. Les liaisons pointillées fluorescentes tracent l'allocation courante des flux.
- **Animation des flux physiques** : Le transit des paquets est animé le long des arêtes actives. En cas de déviation d'un agent vers une meilleure réponse, une transition fluide illustre le basculement physique de la charge d'un nœud à un autre.

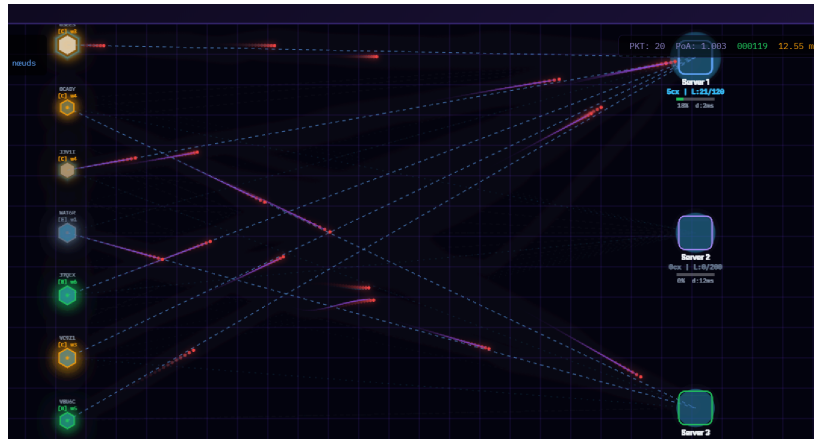


Figure 3.10: Visualisation topologique des flux et routages dynamiques des agents

Tableau de Bord des Métriques et Module d'Exportation CSV

L'onglet dédié aux analyses de performances statistiques (représenté sur la Figure 3.11) regroupe les indicateurs globaux du réseau :

- **Suivi de convergence de la fonction objectif** : Un graphique d'évolution temporelle compare la trajectoire du coût social mesuré (C_{mesur}) par rapport à la barrière théorique de l'optimum social (C_{opt}^*). On y observe distinctement les sauts de coût lors des pannes ou des phases de sur-utilisation.
- **Indicateurs macroscopiques du réseau** : L'interface calcule et affiche le Prix de l'Anarchie ($PoA = C_{Nash}/C_{opt}$) qualifié par un indicateur de régime critique (*Haute Anarchie*), le *Makespan*, le taux d'utilisation globale de l'infrastructure, ainsi que la valeur exacte du Potentiel de Rosenthal $\Phi(s)$.
- **Extraction et exportation de données (CSV)** : Afin de permettre des analyses approfondies et de tracer des courbes de performance sous d'autres environnements de calcul, un bouton **TELECHARGER CSV** est intégré. Ce module extrait instantanément l'intégralité de la télémétrie courante (profils des clients, charges des serveurs, coûts unitaires et historiques du PoA) sous forme de fichiers de données structurés.

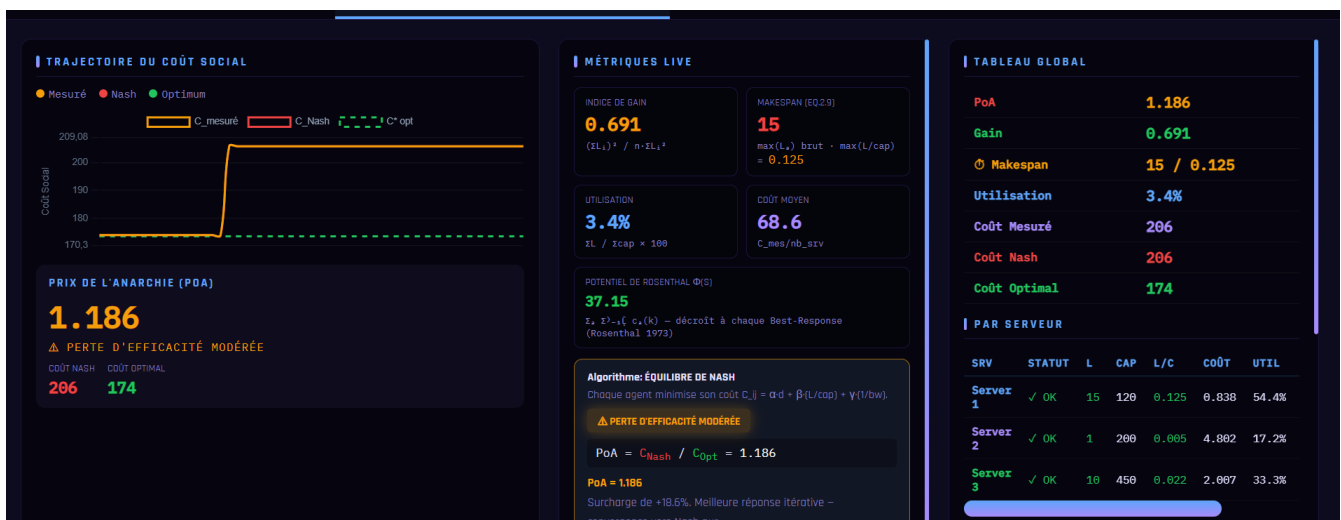


Figure 3.11: Tableau de bord statistique, métriques live du PoA et module d'extraction CSV

Conclusion Générale

Ce projet de fin d'études a été guidé par un objectif central : concevoir et évaluer une approche distribuée, dynamique et résiliente pour la gestion et l'allocation des charges au sein d'architectures réseau modernes. En nous appuyant sur les fondements de la théorie des jeux algorithmique et de l'optimisation combinatoire, nous avons pu modéliser les serveurs non plus comme de simples réceptacles passifs, mais comme les composants d'un écosystème hautement stratégique où s'affrontent et collaborent des flux de données hétérogènes.

Dans un premier temps, l'étude approfondie de la modélisation tri-critère — intégrant simultanément la latence physique, le ratio charge/capacité et la bande passante — nous a permis de dépasser les limites des métriques de routage traditionnelles. L'implémentation de la dynamique de meilleure réponse à travers le potentiel exact de Rosenthal a fourni la preuve théorique de la stabilité du système, garantissant une convergence systématique vers un Équilibre de Nash Pur.

Cependant, la confrontation aux réalités des réseaux réels nous a poussés à explorer des paradigmes plus flexibles. L'intégration de l'Équilibre de Réponse Quantale (QRE) a permis d'introduire avec succès le concept de rationalité limitée, traduisant mathématiquement le bruit, les approximations et les comportements exploratoires des agents face à un environnement changeant. De même, la confrontation des équilibres égoïstes à l'optimum social, formalisée par l'analyse fine du Prix de l'Anarchie (*PoA*), a mis en lumière le coût de l'absence de coordination et l'intérêt d'algorithmes consensuels ou locaux de type Pareto.

Sur le plan pratique, la concrétisation de ces concepts à travers le développement d'une plateforme de simulation interactive en temps réel constitue une des contributions majeures de ce travail. Au-delà de la simple validation visuelle du graphe biparti et du transit des flux, l'interface s'est révélée être un véritable outil d'expérimentation. Les tests de tolérance aux pannes, matérialisés par la désactivation à chaud de serveurs, ainsi que les modules d'extraction de données télémétriques en format CSV, démontrent la viabilité opérationnelle de nos algorithmes de rééquilibrage dynamique.

En perspective, ce travail ouvre la voie à plusieurs extensions prometteuses. Il serait particulièrement intéressant d'introduire des mécanismes d'apprentissage par renforcement (notamment le *Multi-Agent Reinforcement Learning*) pour permettre aux clients d'apprendre leurs stratégies de routage de façon totalement autonome et asynchrone, sans nécessiter de connaissance globale sur l'état des serveurs. Enfin, tester ces modèles sur des topologies réelles à grande échelle, telles que des infrastructures d'Edge Computing ou des réseaux de distribution de contenu (CDN) hautement volatils, permettrait de confronter la robustesse de notre approche tri-critère aux contraintes de production industrielles.

